

ML Final Part A Cheatsheet - Page 1

W8 Trees / Ensembles

Model	Idea	Output
Class tree	Split X; leaf=majority class.	<code>DecisionTreeClassifier(max_depth=6);</code> score=accuracy
Reg tree	Leaf=mean y.	MSE low good; RMSE=sqrt(MSE)
Prune	Limit complexity.	max_depth, min_samples_leaf, deep=overfit
Bagging	Bootstrap many trees, all predictors.	<code>RandomForestRegressor(max_features=</code>
RF	Bagging + random predictors.	<code>feature_importances_;</code> lowers variance
Boosting	Sequential trees learn errors.	accurate; high n_estimators/lr may overfit

```
tree=DecisionTreeClassifier(max_depth=6).fit(Xtr,ytr)
pred=tree.predict(Xte); confusion_matrix(yte,pred);
classification_report(yte,pred)
rf=RandomForestRegressor(max_features=4).fit(Xtr,ytr)
mean_squared_error(yte,rf.predict(Xte));
rf.feature_importances_
```

W9 SVM

Term	Meaning
Hyperplane	Decision boundary; linear SVM=straight.
Margin	Distance to closest points; larger often better.
Support vectors	Closest points determining boundary.
C	Small C=wider margin, more train errors. Large C=fewer train errors, overfit risk.
gamma	RBF locality; large gamma=wiggly/local.
Kernel	linear/rbf/poly; RBF for nonlinear.

```
svc=SVC(C=1, kernel="linear").fit(Xtr,ytr);
svc.score(Xte,yte)
```

```
params={"C": [.01, .1, 1, 10, 100], "gamma": [.5, 1, 2, 3, 4]}
clf=GridSearchCV(SVC(kernel="rbf"), params, cv=10, scoring="accuracy")
clf.fit(Xtr,ytr)
clf.best_params_; clf.best_score_
```

ROC: top-left good. AUC 1 perfect, 0.5 random. Train high + test low = overfit.

W10 PCA

Term	Meaning
Unsupervised	Only X, no y; find structure.
PCA	PCs=linear combinations of original variables.
PC1/PC2	PC1 largest variance; PC2 next; PCs uncorrelated.
Scores	Observation coordinates; plot observations.
Loadings	Variable weights; large abs=influential; sign=direction.
PVE/Scree	Variance explained; cumulative PVE total captured; elbow useful.

```
ys=pd.DataFrame(scale(df), index=df.index, columns=df.columns)
pca=PCA(); scores=pca.fit_transform(Xsc)
loadings=pd.DataFrame(pca.components_.T, index=df.columns)
pve=pca.explained_variance_ratio_; np.cumsum(pve)
```

Scale before PCA if variables have different units.

W11 Clustering

Method	Data	Center	Output
K-means	numeric	mean	labels_ centers_ inertia_
K-modes	categorical	mode	labels_ centroids_ cost_
K-medoids	numeric/dissim	actual point	robust outliers
K-prototypes	mixed	mean+mode	categorical=[cols]

```
km=KMeans(n_clusters=3, n_init=20, random_state=123).fit(X)
km.labels_; km.cluster_centers_; km.inertia_ # lower better
md=KModes(n_clusters=4, init="Huang", n_init=5).fit(data)
md.labels_; md.cluster_centroids_; md.cost_ # lower better
kp=KPrototypes(n_clusters=3, init="Cao");
kp.fit_predict(df, categorical=[0,1])
```

Manual: Euclidean=sqrt(sum squared diffs); Manhattan=sum abs diffs.
K-means: centroid=column means -> nearest -> update -> stop if unchanged.
K-modes: same=0, diff=1, sum -> lowest -> update mode -> state ties.
Example O(A,L,M), C(A,L,C): 0+0+1=1.

W12 Hierarchical / EM

Item	Meaning
Hierarchical	Dendrogram; no K first.
Agglomerative	Bottom-up; each obs starts alone.
Height	Merge dissimilarity; lower=similar; vertical axis matters.
Complete	farthest pair; compact.
Single	closest pair; chaining.
Average	average pairwise distance.
EM/GMM	soft clustering; E probs, M updates.

```
hc=linkage(X, method="complete", metric="euclidean")
dendrogram(hc);
labels=cut_tree(hc, n_clusters=4).reshape(-1)
gmm=GaussianMixture(n_components=2).fit(X);
gmm.predict(X); gmm.predict_proba(X)
```

W13 Neural Nets

Term	Meaning
Perceptron	one neuron; simple linear boundary.
MLP	hidden layers; nonlinear.
Feed-forward	input -> weights/bias -> activation -> output.
Backprop	updates weights from error.
Activation	ReLU/tanh/sigmoid nonlinearity.
Scaling	essential. hidden_layer_sizes=(10,10,10).

```
sc=StandardScaler(); Xtr=sc.fit_transform(Xtr);
Xte=sc.transform(Xte)
mlp=MLPClassifier(hidden_layer_sizes=(10,10,10), max_iter=1000).fit(Xtr,ytr.values.ravel())
confusion_matrix(yte, mlp.predict(Xte));
classification_report(yte, mlp.predict(Xte))
reg=MLPRegressor(activation="relu", hidden_layer_sizes=(16,)).fit(Xtr,ytr)
mean_squared_error(yte, reg.predict(Xte));
r2_score(yte, reg.predict(Xte))
```

ML Final Part A Cheatsheet - Page 2

Output Interpretation

Output	Meaning	Say
.score classifier	accuracy	higher test better
.score regressor	R2	1 perfect, 0 mean, negative bad
confusion matrix	counts	diagonal correct
precision	TP/(TP+FP)	low=many false positives
recall	TP/(TP+FN)	low=many false negatives
F1	balance	good for imbalance
support	true count	check imbalance
MSE/RMSE	error	lower better; RMSE y units
coef	effect	sign=direction
P> t	p-value	<.05significant
R2/adj R2	fit	adj better compare p
AIC/BIC	criteria	lower better; BIC harsher
CV	test estimate	choose best CV

Confusion matrix:

	Pred0	Pred1
Actual0	TN	FP
Actual1	FN	TP

Acc=(TP+TN)/N; Precision=TP/(TP+FP); Recall=TP/(TP+FN)

Core Formulas

Linear: $y=b_0+b_1x_1+\dots+b_px_p+e$
 Residual=actual-pred; $RSS=\sum(\text{resid}^2)$; $MSE=\text{mean}(\text{resid}^2)$;
 $RMSE=\sqrt{MSE}$

Logistic: $p=\exp(\eta)/(1+\exp(\eta))$, $\eta=b_0+b_1x$; odds= $p/(1-p)$; log-odds= η
 Euclidean= $\sqrt{(a_1-b_1)^2+(a_2-b_2)^2+\dots}$; Manhattan= $|a_1-b_1|+|a_2-b_2|+\dots$
 PVE=PC variance/total variance; K-means inertia=sum squared distance to own centroid

Week 1-7 Safety Net

Week	High-yield idea	Command
W1	Reg quantitative y; class categorical y; clustering/PCA no y. Test estimates future.	train_test_split, LinearRegression, KMeans, PCA
W2	Coef sign; p-value relation; residuals/outliers/leverage. Polynomial still linear in beta.	smf.ols(...).summary()
W3	Logistic probability -> threshold -> class. LDA Gaussian/common covariance.	LogisticRegression, LDA, confusion_matrix
W4	Validation/LOOCV/k-fold estimate test error. Bootstrap replacement for uncertainty.	cross_val_score, LeaveOneOut, resample
W5	Best subset all; forward/backward search. Ridge shrinks; lasso zeros variables.	RidgeCV, LassoCV; lower CV/AIC/BIC
W6	PCR=PCA then regression, choose PCs by CV. PLS supervised uses y.	PCA, LinearRegression, CV MSE
W7	Polynomial, step, splines, local reg, GAM. Choose complexity by p-values/CV.	PolynomialFeatures, pd.cut, dmatrix("bs"), anova_lm

Common Commands

```
from sklearn.model_selection import
train_test_split, cross_val_score, GridSearchCV
from sklearn.metrics import
confusion_matrix, classification_report, mean_squared_error,
r2_score
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=.2, random_s
tate=0)
```

```
res=smf.ols("medv~lstat+rm", data=df).fit(); res.summary()
lr=LinearRegression().fit(Xtr, ytr); lr.coef_;
lr.intercept_
log=LogisticRegression(C=100000, tol=1e-7).fit(Xtr, ytr)
scores=cross_val_score(model, X, y, cv=10, scoring="accuracy")
errs=-
cross_val_score(model, X, y, cv=10, scoring="neg_mean_squared_
error")
ridge=RidgeCV(alphas=np.logspace(-3, 3, 100)).fit(Xtr, ytr)
lasso=LassoCV(alphas=np.logspace(-3, 3, 100), max_iter=100000
).fit(Xtr, ytr)
lasso.coef_ # zeros=removed
X4=PolynomialFeatures(degree=4).fit_transform(X)
spl=dmatrix("bs(age, df=6, include_intercept=False)",
{"age": df.age}, return_type="dataframe")
```

Fast Comparisons

Compare	Answer
train vs test	train optimistic; test unseen performance
bias/variance	simple high bias low var; flexible low bias high var
bagging/RF	bagging all predictors; RF random subset
RF/boosting	independent vs sequential trees
PCA/clustering	reduce/visualize vs group observations
K-means/hier	need K first vs dendrogram then cut
K-means/K-modes	numeric mean vs categorical mode
hard/soft	one label vs membership probabilities
ridge/lasso	shrink vs shrink + select
PCR/PLS	PCR ignores y for PCs; PLS uses y

Answer Frames

Output: Test ____ Means ____, Better/worse than ____ because ____ Train>>test => overfit.
 Select model: lower test/CV MSE or higher test/CV accuracy/F1; lower AIC/BIC.
 K-means: centroid -> distance -> nearest -> update -> stop unchanged.
 K-modes: same0/diff1 -> sum -> lowest -> update mode -> mention ties.